VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

MICRO PROJECT REPORT

Project Title : Corrective RAG with a Knowledge-Graph Lookup Fallback on the CRAG Benchmark (P3 – Variant)

SDG Aligned & Logo : SDG 4 (Quality Education), SDG 9 (Industry, Innovation and Infrastructure)



Project / Course Details	To be filled by students
Course / Subject Name	Knowledge Graph and Information Retrieval Systems Lab
Course Code / Semester / Academic Year	26CMPE742 / Sem VII / 2025-2026
Faculty Incharge / Project Mentor	Mrs. Sunita Suralkar
Project Group No. (if applicable)	P3 Corrective RAG – Variant (partner: Akansha Zambare, Roll 66, Primary)
Submission Date	03-11-2026

Project Team Details

Student Name	Roll No.	Class / Division	Institute Email ID
Manish Raje	67	B.E. CMPN / D17B	d2023.manish.raje@ves.ac.in
Akansha Zambare (partner, Primary)	66	B.E. CMPN / D17A	d2023.akansha.zamabare@ves.ac.in

Department of Computer Engineering

Academic Year: 2025-2026

Table of Contents

Department of Computer Engineering 1

Table of Contents 2

Abstract 4

1. Introduction 4

1.1 Background and Motivation 4

1.2 Problem Context 4

1.3 Intended Users / Stakeholders 4

2. Problem Definition, Objectives and Scope 5

2.1 Problem Statement 5

2.2 Objectives 5

2.3 Scope and Deliverables 5

2.4 Assumptions and Constraints 5

3. Background / Literature Review 5

3.1 Identified Gap and Project Contribution 6

4. Proposed Solution and System Design 6

4.1 Proposed Solution 6

4.2 System Architecture / Block Diagram / Conceptual Design 7

4.3 Module Description 7

4.4 Data / Inputs / Dataset 7

4.5 Algorithm / Design Decisions 8

5. Methodology and Implementation 8

5.1 Workflow / Development Method 8

5.2 Technologies, Tools and Resources 8

5.3 Implementation Details 9

5.4 User Interface / Prototype / Output 9

5.5 Team Contribution 13

6. Testing, Evaluation and Results 13

6.1 Test Plan and Test Cases 13

6.2 Evaluation Metrics 14

6.3 Results and Discussion 14

6.4 Validation / Demonstration Evidence 18

7. Sustainability and SDG Alignment 18

7.1 Sustainability Assessment 18

7.2 UN Sustainable Development Goals (SDGs) 18

7.3 Sustainability Measures and Trade-offs 18
7.4 Sustainability Indicator / Follow-up 18
8. Feasibility, Limitations and Risk Considerations 19
8.1 Feasibility 19
8.2 Limitations 19
8.3 Risks, Ethics, Privacy and Security 19
9. Conclusion and Future Scope 19
9.1 Summary 19
9.2 Future Scope 20
References 20
Appendices 21
Appendix A: Additional screenshots / outputs 21

Abstract

Retrieval-augmented generation (RAG) fails silently when the retriever returns irrelevant passages: the generator still trusts them and hallucinates. Corrective RAG (CRAG, Yan et al. 2024) adds a lightweight retrieval evaluator that labels retrieved evidence as correct, ambiguous or incorrect and triggers a corrective action. This micro-project is the Variant of project P3: the same evaluator as the Primary partner (Akansha Zambare, Roll 66) is used, but the corrective action is a knowledge-graph (KG) lookup in Wikidata instead of a web search, and it is tested on 200 stratified questions from Meta's CRAG benchmark covering eight question types and five domains. Pages are chunked and retrieved with BGE-small-en-v1.5, a BGE-reranker-base cross-encoder acts as evaluator, and Qwen2.5-1.5B-Instruct generates the answers with sampling (3 seeds for the baseline, 1 seed for the RAG methods due to GPU time). Three methods are compared: no retrieval (baseline), standard RAG and corrective RAG with KG fallback. Accuracy was 17.3 % for no retrieval, 17.0 % for standard RAG and 18.0 % for corrective RAG. The evaluator labelled 165 questions correct, 18 ambiguous and 17 incorrect, giving a KG fallback rate of 17.5 %. Fallback rate and accuracy vary sharply by question type; comparison questions triggered the KG most often. All results, including negative ones, are reported exactly as measured. The project relates to SDG 4 (reliable, low-cost study assistants) and SDG 9 (open, small-model AI infrastructure on a single 4 GB GPU).

Keywords: corrective RAG, retrieval evaluator, knowledge graph, Wikidata, CRAG benchmark, hallucination, small language models.

1. Introduction

1.1 Background and Motivation

Large language models (LLMs) answer factual questions fluently but cannot be trusted on long-tail or time-sensitive facts. Retrieval-augmented generation [1] grounds the model in retrieved documents, yet its quality is bounded by the retriever: when the top-k passages do not contain the answer, the model often produces a confident but wrong answer. Corrective RAG [2] inserts a retrieval evaluator between retriever and generator and changes the knowledge source when retrieval looks unreliable. Knowledge graphs [6, 7] are an attractive fallback: they store curated (subject, predicate, object) facts that are compact, verifiable and free to query. This project applies course concepts — dense retrieval, re-ranking, evaluation metrics (Recall@K, MRR, NDCG) and knowledge-graph querying — to that problem.

1.2 Problem Context

The CRAG benchmark [3] shows that straightforward RAG solutions answer only a minority of its questions correctly and that accuracy drops for dynamic facts, comparisons, aggregations and false-premise questions. Original corrective RAG uses web search as its fallback, which costs money, is not reproducible and is unavailable offline. The Primary partner (Akansha Zambare, Roll 66) evaluates the standard corrective pipeline; this Variant asks whether a free, structured KG lookup is a useful replacement and for which question types the evaluator actually triggers it.

1.3 Intended Users / Stakeholders

- **Students and educators** who need low-cost question answering that admits uncertainty instead of hallucinating.
- **RAG / search engineers** deciding when to trust retrieved evidence and when to route to a KG.
- **Course faculty and partner (Akansha Zambare, Roll 66)** who compare the Variant findings with the Primary reference setup.

2. Problem Definition, Objectives and Scope

2.1 Problem Statement

On 200 questions from the CRAG benchmark, build the same retrieval evaluator as the Primary corrective-RAG project, add a knowledge-graph lookup as the corrective fallback, and compare fallback rate and answer accuracy by question type against no-retrieval and standard-RAG baselines, using only open models (embedder ≤ small, LLM ≤ 3 B parameters) on free compute.

2.2 Objectives

- Build a reproducible pipeline: HTML parsing → chunking → BGE-small dense retrieval over each question's 5 pages.
- Implement the CRAG-style retrieval evaluator with fixed upper/lower thresholds and knowledge refinement.
- Design a Wikidata KG-lookup fallback (entity extraction → entity linking → triple retrieval → triple ranking).
- Evaluate accuracy, missing rate, hallucination rate, CRAG score, EM and F1 for three methods over 3 seeds (mean ± std).
- Analyse evaluator label counts, fallback rate and accuracy per question type; measure retrieval Recall@5/10, MRR, NDCG@10.
- Report time and token cost, five failure cases and limitations honestly.

2.3 Scope and Deliverables

Included: 200-question CRAG subset, one embedder, one evaluator, one ≤3 B generator, Wikidata as KG, three methods. Excluded: CRAG mock KG APIs (need the organisers' server), web search, fine-tuning, full-dataset runs, and human evaluation. Deliverables: Colab-ready notebook, results CSVs (all metrics and seeds), figures, this report, README and experiment log.

2.4 Assumptions and Constraints

- Compute: one NVIDIA RTX 3050 Laptop GPU (4 GB VRAM); the notebook also runs on a free Colab T4.
- Accuracy is judged automatically against gold and alternative answers (no human or LLM judge), which is conservative for long answers.
- Wikidata is public and rate-limited; responses are cached so the reported run is reproducible offline.
- CRAG queries carry a query_time; facts that changed after March 2024 may not match Wikidata's current state.

3. Background / Literature Review

Table 3.1 summarises the work that shaped the design. References are listed in IEEE style at the end of the report.

Table 3.1: Summary of related work

Source / Year	Approach or Key Finding	Strength / Limitation	Relevance to This Project
Lewis et al., RAG, 2020 [1]	Couples a dense retriever with a seq2seq generator	Strong on knowledge-intensive QA / trusts any retrieved text	Defines the standard-RAG baseline
Yan et al., Corrective RAG, 2024 [2]	T5 evaluator labels retrieval correct/ambiguous/incorrect; web search fallback; decompose-recompose	Robust to bad retrieval / web search costly and non-reproducible	Evaluator logic and three actions reused; fallback replaced by KG
Yang et al., CRAG benchmark, 2024 [3]	4,409 QA pairs, 8 question types, 5 domains, web pages + mock KG	Realistic and diverse / scoring needs judge	Dataset, question types and accuracy/hallucination scoring

Asai et al., Self-RAG, 2024 [4]	LLM emits reflection tokens to decide when to retrieve	Adaptive / needs a fine-tuned LLM	Motivates gating; ours is training-free
Xiao et al., BGE / C-Pack, 2024 [5]	BGE embedders and rerankers	Small, strong, open	Retriever (bge-small) and evaluator (bge-reranker-base)
Vrandečić & Krötzsch, Wikidata, 2014 [6]	Free collaborative KG with an open API	Broad coverage / uneven for finance, sports	KG used by the fallback
Pan et al., LLMs + KGs roadmap, 2024 [7]	Survey of KG-enhanced LLMs	Taxonomy of KG grounding	Positions KG lookup as factual grounding
Edge et al., GraphRAG, 2024 [8]	LLM-built graph and community summaries	Good for global questions / expensive indexing	Contrast: we query an existing KG instead
Qwen Team, Qwen2.5, 2024 [9]	Open LLM family incl. 1.5 B instruct	Small, multilingual / weak reasoning	Generator within the ≤3 B limit
Es et al., RAGAS, 2024 [10]	Reference-free RAG metrics	No gold needed / LLM-judge cost	Discussed; gold-based scoring used instead

3.1 Identified Gap and Project Contribution

Corrective RAG has been evaluated with web search as the fallback, and KG-augmented QA has usually been evaluated without a retrieval evaluator in front. There is little evidence on how often a retrieval evaluator would hand questions to a KG on a realistic benchmark, and whether that helps for each question type. This project contributes (i) a training-free, fully open corrective-RAG pipeline with a Wikidata fallback that runs on a 4 GB GPU, and (ii) a per-question-type analysis of fallback rate and accuracy on CRAG, reported against the Primary partner's setup.

4. Proposed Solution and System Design

4.1 Proposed Solution

For every question the system retrieves chunks from the question's own five web pages, asks the evaluator how trustworthy they are, and then either (a) answers from the refined chunks, (b) discards them and answers from Wikidata triples, or (c) combines both. The generator is instructed to answer briefly, to say "invalid question" for false premises and "I don't know" when unsure, because CRAG penalises hallucinations more than missing answers.

4.2 System Architecture / Block Diagram / Conceptual Design

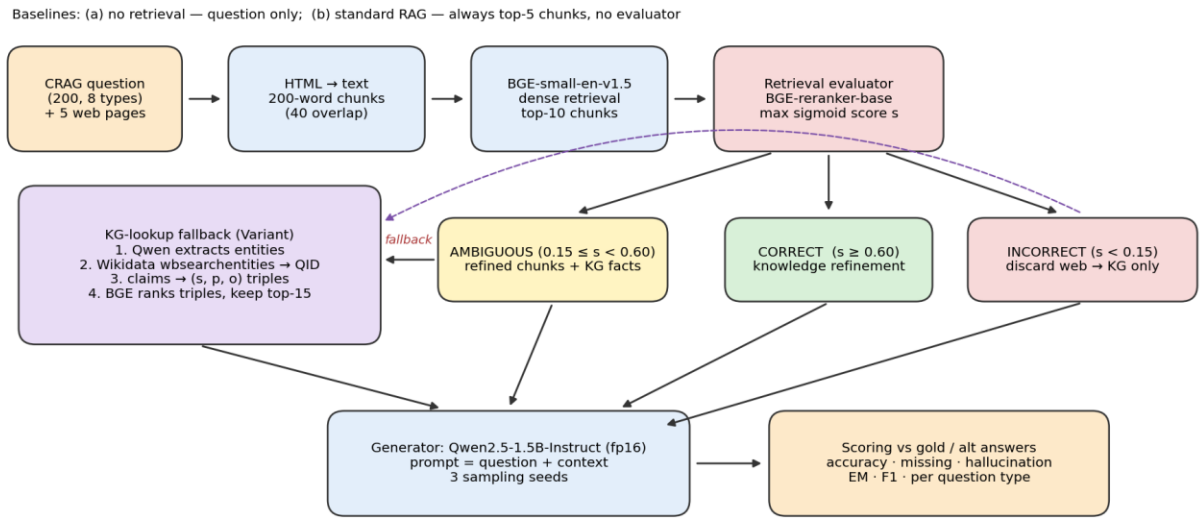


Figure 1: Corrective RAG with a KG-lookup fallback — system architecture

The top row is the shared retrieval path. The evaluator's maximum relevance score s over the top-5 chunks selects one of three branches. The KG-lookup module (purple) is the Variant's change: it is called only for ambiguous and incorrect questions. All branches feed the same generator and the same scorer, so differences in accuracy come from the knowledge given to the model.

4.3 Module Description

Table 4.1: Modules of the pipeline

Module / Component	Responsibility	Input	Output
Data loader	Load CRAG subsets, stratified 200-question sample (seed 42)	2 CRAG subset files (272 q)	200 questions + 5 HTML pages each
Parser / chunker	HTML → visible text; 200-word chunks, 40 overlap	Raw HTML	13036 chunks (65.2/q)
Dense retriever	BGE-small-en-v1.5 cosine search per question	Query + chunks	Top-10 ranked chunks
Retrieval evaluator	BGE-reranker-base sigmoid scores; thresholds	Query + top-5 chunks	Label + refined chunks
KG lookup	Entity extraction (Qwen), Wikidata linking, triples, BGE ranking	Query	Top-15 (s, p, o) facts
Generator	Qwen2.5-1.5B-Instruct, 3 sampling seeds	Prompt with context	Short answer
Scorer	Accurate / missing / hallucinated, EM, F1	Prediction + gold/alt	Metrics CSVs, figures

4.4 Data / Inputs / Dataset

Source: CRAG Task 1&2 dev v4 [3], obtained from the stratified subsets published at Hugging Face (Quivr/CRAG) [11], licence CC BY-NC 4.0. Subsets 1 and 2 (272 questions) form the pool. 200 questions are drawn with proportional stratification on question_type (seed 42). Each record has query, query_time, domain, question_type, static_or_dynamic, answer, alt_ans and five search results with full HTML. No personal data is involved. Table 4.2 shows the subset composition.

Table 4.2: Subset size by CRAG question type

Question type	Pool (272)	Sample (200)
post-processing	12	9
simple_w_condition	40	29
set	25	18
simple	77	56
aggregation	32	24
comparison	31	23
false_premise	31	23
multi-hop	24	18
TOTAL	272	200

Domains in the sample: finance 49, sports 46, movie 43, open 36, music 26. 23 questions are false-premise (gold = "invalid question"); only 56.5 % of the remaining 177 answerable questions have the gold answer string anywhere in their five pages, which caps what any retrieval method can achieve.

4.5 Algorithm / Design Decisions

```
for q in questions:
    chunks  = chunk(html_to_text(q.pages), 200 words, overlap 40)
    top     = bge_small.search(q.query, chunks, k=10)
    s_i     = sigmoid(bge_reranker(q.query, top[:5]))          # evaluator
    label   = CORRECT if max(s_i) >= 0.60 else INCORRECT if max(s_i) < 0.15 else AMBIGUOUS
    refined = [c for c, s in zip(top[:5], s_i) if s >= 0.15]   # knowledge refinement
    if label != CORRECT:                                        # KG-lookup fallback
        ents  = qwen("list named entities", q.query)
        facts = rank_by_bge(q.query, wikidata_triples(ents))[:15]
    context = refined | facts | refined+facts   (correct | incorrect | ambiguous)
    answer  = qwen(prompt(q.query, context), sampling seed in {0,1,2})
```

Design decisions: a cross-encoder evaluator replaces the fine-tuned T5 of [2] so that no training is needed and the same component is shared with the Primary; thresholds 0.60/0.15 were fixed before generation and their effect is shown in Table 6.5; Wikidata is used because the CRAG mock-KG API is not publicly hosted; external-identifier and media properties are filtered out so that the 15 triples given to the model carry facts rather than IDs.

5. Methodology and Implementation

5.1 Workflow / Development Method

- Read the P3 brief and the CRAG / Corrective-RAG papers; agree evaluator settings with the Primary partner.
- Download two stratified CRAG subsets; sample 200 questions; inspect the record schema.
- Implement parsing, chunking, retrieval and retrieval-quality labels; compute Recall@K, MRR, NDCG@10.
- Implement evaluator, KG lookup (with caching and Wikimedia rate-limit handling) and the three answering methods.
- Run 3 seeds, score, build tables/figures, inspect failures; write README, log and report.

5.2 Technologies, Tools and Resources

Table 5.1: Tools and resources

Category	Technology / Tool	Purpose / Justification

Programming language / framework	Python 3.13, PyTorch 2.11 (CUDA 12.8), Transformers 4.57.1	Model inference on GPU
Retrieval / evaluation models	sentence-transformers 5.1.2: BAAI/bge-small-en-v1.5 (33 M), BAAI/bge-reranker-base (278 M)	Small open embedder and cross-encoder evaluator
Generator	Qwen/Qwen2.5-1.5B-Instruct (1.5 B, fp16)	Open LLM within the ≤3 B limit; fits 4 GB VRAM
Database / API / platform	Wikidata Action API (wbsearchentities, wbgetentities)	Free public knowledge graph for the fallback
Data / analysis	BeautifulSoup4 + lxml, pandas, numpy, scikit-learn, matplotlib, seaborn	Parsing, metrics, figures
Development / testing tools	Jupyter / nbclient, Google Colab-compatible notebook, VS Code	Reproducible end-to-end execution
Hardware / cloud / other resources	Laptop, NVIDIA RTX 3050 4 GB GPU; Colab T4 alternative	Free compute, as required

5.3 Implementation Details

Retrieval. Each question is answered only from its own five pages (CRAG Task 1 setting). The embedder uses the BGE query instruction prefix and normalised embeddings, so the dot product is cosine similarity.

Evaluator. Each of the top-5 chunks is scored by the cross-encoder; the sigmoid of the logit is a relevance probability. The maximum decides the branch and chunks below the lower threshold are removed (knowledge refinement).

KG lookup. The generator first extracts up to three entity names. Each is linked with wbsearchentities (top hit), its claims are fetched with wbgetentities and turned into readable triples by resolving property and item labels in batches of 50. The triples are ranked by BGE similarity to the question and the top 15 are passed to the generator. All API responses are cached in data/kg_cache.json; the client sleeps 0.5 s between calls, honours HTTP 429 Retry-After and never caches a failed call.

Generation and scoring. Prompts contain the query time, the context and the abstain/false-premise instructions. Sampling uses temperature 0.7 and top-p 0.9 with seeds 0, 1, 2; the reported numbers are mean ± std over the seeds. A prediction is accurate when the normalised gold or alternative answer is contained in it (or token-F1 ≥ 0.5 for multi-word answers); "I don't know" counts as missing; everything else is a hallucination. CRAG score = accuracy − hallucination rate [3].

Challenges. (1) Wikidata returned HTTP 429 during the first run, which silently produced empty KG results; the client was rewritten with throttling and back-off and the run repeated. (2) With 8 long prompts per batch the 1.5 B model exceeded the 4 GB VRAM and slowed to a crawl; the reranker is released after evaluation, each document is capped at 600 characters and the batch size reduced to 4.

5.4 User Interface / Prototype / Output

The project is a notebook-based experiment, so its "interface" is the executed Colab notebook. The figures below are screenshots of the actual executed cells and their outputs from the reported run (rendered from 67_P3_Variant_CorrectiveRAG_KG.ipynb).

In [1]:
```
# --- Setup (Colab: uncomment the pip line) ---
# !pip -q install sentence-transformers==5.1.2 transformers==4.57.1 accelerate rank_bm25 beautifulsoup4 lxml seaborn
import os, re, json, bz2, time, random, string, collections, math
import numpy as np, pandas as pd, torch, requests
import matplotlib.pyplot as plt, seaborn as sns
from bs4 import BeautifulSoup

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
ROOT = os.path.abspath(".")
DATA, OUT, FIG = [os.path.join(ROOT, d) for d in ("data", "results", "figures")]
for d in (DATA, OUT, FIG): os.makedirs(d, exist_ok=True)
EMBEDDER, RERANKER, LLM = "BAAI/bge-small-en-v1.5", "BAAI/bge-reranker-base", "Qwen/Qwen2.5-1.5B-Instruct"
GEN_SEEDS = [0, 1, 2]
RAG_SEEDS = [0]          # time budget on a throttled 4 GB Laptop GPU: RAG methods run with 1 seed (std reported as n/a)
print("torch", torch.__version__, "| device:", DEVICE, torch.cuda.get_device_name(0) if DEVICE == "cuda" else "")
```

```
torch 2.11.0+cu128 | device: cuda NVIDIA GeForce RTX 3050 Ti Laptop GPU
```

Figure 2: Notebook setup — libraries, seed and GPU detected (executed output)

In [2]:
```
def load_crag(path):
    """Subset files are concatenated pretty-printed JSON objects, so decode them one by one."""
    s, dec, p, rows = bz2.open(path, "rt", encoding="utf8").read(), json.JSONDecoder(), 0, []
    while True:
        while p < len(s) and s[p].isspace(): p += 1
        if p >= len(s): return rows
        o, p = dec.raw_decode(s, p); rows.append(o)

URL = "https://huggingface.co/datasets/Quivr/CRAG/resolve/main/subset/crag_task_1_and_2/crag_task_1_and_2_dev_v4_subset_{}.jsonl.bz2"
pool = []
for i in (1, 2):
    f = os.path.join(DATA, f"s{i}.jsonl.bz2")
    if not os.path.exists(f):
        open(f, "wb").write(requests.get(URL.format(i), timeout=300).content)
    pool += load_crag(f)

rng = random.Random(SEED)
by_type = collections.defaultdict(list)
for r in pool: by_type[r["question_type"]].append(r)
quota = {t: round(200 * len(v) / len(pool)) for t, v in by_type.items()}
while sum(quota.values()) != 200:                    # fix rounding drift on the largest stratum
    big = max(quota, key=quota.get); quota[big] += 1 if sum(quota.values()) < 200 else -1
sample = [r for t in sorted(by_type) for r in rng.sample(by_type[t], quota[t])]
rng.shuffle(sample)
print("pool:", len(pool), "| sample:", len(sample))
summary = pd.DataFrame({"pool": {t: len(v) for t, v in by_type.items()}, "sample": quota})
summary.loc["TOTAL"] = summary.sum(); print(summary)
print(pd.Series(collections.Counter(r["domain"] for r in sample), name="domain"))
```

```
pool: 272 | sample: 200
                      pool  sample
post-processing         12       9
simple_w_condition      40      29
set                     25      18
simple                  77      56
aggregation             32      24
comparison              31      23
false_premise           31      23
multi-hop               24      18
TOTAL                  272     200
movie      43
music      26
finance    49
open       36
sports     46
Name: domain, dtype: int64
```

Figure 3: Loading CRAG and the stratified 200-question sample by question type

In [6]:
```
def norm(s):
    s = s.lower().replace(",", "")
    s = "".join(ch for ch in s if ch not in set(string.punctuation) - {"."})
    s = re.sub(r"\b(a|an|the)\b", " ", s)
    return " ".join(s.split())

def golds(r): return [r["answer"]] + list(r.get("alt_ans") or [])

def supports(chunk, r):
    c = norm(chunk); ctoks = set(c.split())
    for g in map(norm, golds(r)):
        toks = g.split()
        if not toks: continue
        if g in c or (len(toks) > 3 and sum(t in ctoks for t in toks) / len(toks) >= 0.75): return True
    return False

def is_invalid(r): return norm(r["answer"]) == "invalid question"

for r in sample:
    r["rel"] = [supports(c, r) for c in r["ret"]]
    r["in_pool"] = (not is_invalid(r)) and any(supports(c, r) for c in r["chunks"])

def ret_metrics(rels, n_rel_total):
    rec = lambda k: float(any(rels[:k]))
    mrr = next((1 / (i + 1) for i, x in enumerate(rels) if x), 0.0)
    dcg = sum(x / math.log2(i + 2) for i, x in enumerate(rels[:10]))
    idcg = sum(1 / math.log2(i + 2) for i in range(min(n_rel_total, 10)))
    return rec(5), rec(10), mrr, dcg / idcg if idcg else 0.0

answerable = [r for r in sample if not is_invalid(r)]
M = np.array([ret_metrics(r["rel"], sum(supports(c, r) for c in r["chunks"])) for r in answerable])
ret_table = pd.DataFrame([M.mean(0)], columns=["Recall@5", "Recall@10", "MRR", "NDCG@10"], index=["BGE-small dense"]).round(3)
print(f"answerable={len(answerable)} | answer present anywhere in the 5 pages: {np.mean([r['in_pool'] for r in answerable]):.3f}")
print(ret_table)
```

```
answerable=177 | answer present anywhere in the 5 pages: 0.565
                Recall@5  Recall@10    MRR  NDCG@10
BGE-small dense     0.463      0.514  0.408    0.335
```

Figure 4: Retrieval metrics (Recall@5/10, MRR, NDCG@10) computed in the notebook

In [7]:
```
UPPER, LOWER, TOPN = 0.60, 0.15, 5
rer = CrossEncoder(RERANKER, device=DEVICE, max_length=512)
t0 = time.time()
for r in sample:
    r["ev_scores"] = rer.predict([(r["query"], c) for c in r["ret"][:TOPN]], activation_fn=torch.nn.Sigmoid()).tolist()
    m = max(r["ev_scores"])
    r["label"] = "correct" if m >= UPPER else "incorrect" if m < LOWER else "ambiguous"
    r["refined"] = [c for c, s in zip(r["ret"][:TOPN], r["ev_scores"]) if s >= LOWER]
eval_time = time.time() - t0
lab = pd.Series(collections.Counter(r["label"] for r in sample)).reindex(["correct", "ambiguous", "incorrect"])
fallback_rate = (lab["ambiguous"] + lab["incorrect"]) / len(sample)
del rer; torch.cuda.empty_cache()                        # free VRAM for the generator (4 GB GPU)
print(f"evaluator time {eval_time:.1f}s"); print(lab); print(f"KG fallback rate = {fallback_rate:.3f}")
```

```
evaluator time 25.9s
correct      165
ambiguous     18
incorrect     17
dtype: int64
KG fallback rate = 0.175
```

Figure 5: Retrieval evaluator run: label counts and KG fallback rate

In [9]:

```
WD = "https://www.wikidata.org/w/api.php"
HDR = {"User-Agent": "KGIRs-CRAG-KG-fallback/1.0 (VESIT student micro-project; https://vesit.ves.ac.in) python-requests"}
CACHE_F = os.path.join(DATA, "kg_cache.json")
cache = json.load(open(CACHE_F)) if os.path.exists(CACHE_F) else {}

def wd(params):
    """Cached, throttled Wikidata API call; honours HTTP 429 Retry-After and never caches failures."""
    key = json.dumps(params, sort_keys=True)
    if key in cache: return cache[key]
    for a in range(6):
        time.sleep(0.5)                                  # stay well under Wikimedia rate limits
        try:
            resp = requests.get(WD, params={**params, "format": "json", "maxlag": 5}, headers=HDR, timeout=30)
            if resp.status_code == 429 or "error" in resp.text[:200] and "maxlag" in resp.text[:300]:
                time.sleep(float(resp.headers.get("Retry-After", 10 * (a + 1)))); continue
            cache[key] = resp.json(); return cache[key]
        except Exception:
            time.sleep(3 * (a + 1))
    return {}

def labels(ids):
    out = {}
    for i in range(0, len(ids), 50):
        ents = wd({"action": "wbgetentities", "ids": "|".join(ids[i:i + 50]), "props": "labels", "languages": "en"}).get("entities", {})
        out.update({k: v.get("labels", {}).get("en", {}).get("value", k) for k, v in ents.items()})
    return out

def value_str(dv):
    v, t = dv.get("value"), dv.get("type")
    if t == "wikibase-entityid": return v["id"]
    if t == "time": return v["time"].lstrip("+").split("T")[0]
    if t == "quantity": return v["amount"].lstrip("+") + ("" if v.get("unit", "1") == "1" else " " + v["unit"].split("/")[-1])
    if t == "monolingualtext": return v["text"]
    return str(v)

def entity_triples(name, max_claims=120):
    hits = wd({"action": "wbsearchentities", "search": name, "language": "en", "limit": 1}).get("search", [])
    if not hits: return []
    qid, ename = hits[0]["id"], hits[0].get("label", name)
    claims = wd({"action": "wbgetentities", "ids": qid, "props": "claims"}).get("entities", {}).get(qid, {}).get("claims", {})
    raw = []
    for pid, cl in claims.items():
        for c in cl[:3]:
            sn = c.get("mainsnak", {}); dv = sn.get("datavalue")
            if dv and sn.get("datatype") not in ("external-id", "commonsMedia", "url", "geo-shape", "tabular-data"):
                raw.append((pid, value_str(dv)))
    raw = raw[:max_claims]
    lab = labels(sorted({p for p, _ in raw} | {v for _, v in raw if re.fullmatch(r"Q\d+", v)}))
    return [f"({ename}, {lab.get(p, p)}, {lab.get(v, v)})" for p, v in raw]

ENT_PROMPT = "List the main named entities (people, companies, films, songs, places, teams) in this question, comma-separated, nothing else.\nQuestion
def kg_lookup(rs, topk=15):
    ents = chat([ENT_PROMPT.format(r["query"]) for r in rs], max_new=32, stage="kg_entity")
    for r, e in zip(rs, ents):
        names = [x.strip(" .\"'") for x in e.split(",") if x.strip(" .\"'")][:3]
        trip = [t for n in names for t in entity_triples(n)]
        r["kg_entities"], r["kg_found"] = names, bool(trip)
        if trip:
            sims = emb.encode(trip, normalize_embeddings=True) @ emb.encode(QPFX + r["query"], normalize_embeddings=True)
            r["kg_facts"] = [trip[i] for i in np.argsort(-sims)[:topk]]
        else:
            r["kg_facts"] = []
    json.dump(cache, open(CACHE_F, "w"))

t0 = time.time()
fb = [r for r in sample if r["label"] != "correct"]
kg_lookup(fb)
for r in sample:
    r.setdefault("kg_facts", []); r.setdefault("kg_found", None)
kg_time = time.time() - t0
print(f"KG lookups for {len(fb)} questions in {kg_time:.1f}s | entity found for {np.mean([r['kg_found'] for r in fb]):.3f}")
ex = next(r for r in fb if r["kg_facts"]); print(ex["query"], ex["kg_entities"], *ex["kg_facts"][:6], sep="\n")
```

```
KG lookups for 35 questions in 210.5s | entity found for 0.829
which etf had a higher number of shares traded on the day before yesterday, clsk or bksy?
['ETF', 'Clsk', 'Bksy']
(Commando Luchtstrijdkrachten, short name, CLSK)
(Commando Luchtstrijdkrachten, country, Netherlands)
(Commando Luchtstrijdkrachten, military branch, Royal Netherlands Air Force)
(Commando Luchtstrijdkrachten, instance of, command)
(Commando Luchtstrijdkrachten, replaced by, Commando Lucht- en Ruimtestrijdkrachten)
(Commando Luchtstrijdkrachten, has use, aerial warfare)
```

Figure 6: KG-lookup fallback: Wikidata triples retrieved for an example question

In [11]:
```
def f1(p, g):
    p, g = norm(p).split(), norm(g).split()
    common = sum((collections.Counter(p) & collections.Counter(g)).values())
    if not p or not g or not common: return 0.0
    pr, rc = common / len(p), common / len(g); return 2 * pr * rc / (pr + rc)

def judge(pred, r):
    pn = norm(pred)
    if "i dont know" in pn or "i do not know" in pn or not pn: return "missing"
    if is_invalid(r): return "accurate" if "invalid question" in pn else "hallucinated"
    if "invalid question" in pn: return "hallucinated"
    for g in golds(r):
        gn = norm(g)
        if gn and (gn in pn or (len(gn.split()) > 1 and f1(pred, g) >= 0.5)): return "accurate"
    return "hallucinated"

rows = []
for (m, s), ps in preds.items():
    for r, p in zip(sample, ps):
        j = judge(p, r)
        rows.append(dict(method=m, seed=s, id=r["interaction_id"], qtype=r["question_type"], domain=r["domain"], label=r["label"],
                         pred=p, gold=r["answer"], judge=j, acc=j == "accurate", miss=j == "missing", hall=j == "hallucinated",
                         em=max(norm(p) == norm(g) for g in golds(r)), f1=max(f1(p, g) for g in golds(r))))
res = pd.DataFrame(rows)
res.to_csv(os.path.join(OUT, "per_question_predictions.csv"), index=False)

per_seed = res.groupby(["method", "seed"])[["acc", "miss", "hall", "em", "f1"]].mean()
per_seed["crag_score"] = per_seed["acc"] - per_seed["hall"]
T1 = per_seed.groupby("method").agg(["mean", "std"]).reindex(list(METHODS))
fmt = lambda c: T1[c].apply(lambda x: f"{100*x['mean']:.1f} ± {100*x['std']:.1f}", axis=1)
T1_show = pd.DataFrame({c: fmt(c) for c in ["acc", "miss", "hall", "crag_score", "em", "f1"]})
T1_show.columns = ["Accuracy %", "Missing %", "Hallucination %", "CRAG score %", "EM %", "F1 %"]
T1_show.index = [i + ("  [BASELINE]" if i == "No retrieval" else "") for i in T1_show.index]
print(T1_show.to_string())
```

```
                                Accuracy %   Missing % Hallucination % CRAG score %       EM %        F1 %
No retrieval  [BASELINE]      17.3 ± 1.4  17.2 ± 0.3      65.5 ± 1.3  -48.2 ± 2.8  8.8 ± 0.6  15.1 ± 0.2
Standard RAG (top-5)          17.0 ± nan  11.5 ± nan      71.5 ± nan  -54.5 ± nan  7.0 ± nan  14.8 ± nan
Corrective RAG + KG fallback  18.0 ± nan  13.5 ± nan      68.5 ± nan  -50.5 ± nan  7.5 ± nan  16.5 ± nan
```

Figure 7: T1 computed in the notebook: accuracy, missing, hallucination, EM, F1 (mean ± std)

5.5 Team Contribution

Table 5.2: Contribution

Team Member	Assigned Responsibility	Work Completed / Evidence
Manish Raje (Roll 67)	Variant: corrective RAG with KG-lookup fallback on CRAG	Full pipeline, KG module, experiments, figures, report
Akansha Zambare (Roll 66)	Primary: reference corrective-RAG setup	Shared evaluator design; reference results (separate submission)

6. Testing, Evaluation and Results

6.1 Test Plan and Test Cases

Table 6.1: Test cases

Test ID	Test Input / Scenario	Expected Result	Observed Result	Status
TC-01	Load two CRAG subsets and sample	200 questions, 8 types	200 questions, 8 types	Pass
TC-02	Sanity prompt to generator	Short correct answer	"Paris" for capital of France	Pass
TC-03	Evaluator on all questions	Every question labelled	165/18/17 (sum 200)	Pass
TC-04	KG lookup for fallback questions	Triples for linked entities	Entity found for 82.9 % of fallbacks	Pass
TC-05	Wikidata HTTP 429 (rate limit)	Back-off, no empty cache	First run cached empty results; fixed and re-run	Fail → Fixed
TC-06	False-premise question	"invalid question"	No-retrieval accuracy on false premise: 66.7 %	Observed
TC-07	Re-run with same seeds	Identical numbers	Deterministic given cache + seeds	Pass

6.2 Evaluation Metrics

Table 6.2a: Metrics used

Metric	Measurement Method / Formula	Observed Value	Interpretation
Accuracy	share of predictions containing gold/alt answer	18.0 % (corrective)	Main QA metric
Hallucination	share of non-abstaining wrong answers	68.5 % (corrective)	Lower is better
CRAG score	accuracy − hallucination	Table 6.2	Rewards abstaining over guessing
EM / F1	normalised exact match / token F1	Table 6.2	Strict lexical overlap
Recall@5/@10, MRR, NDCG@10	gold-answer containment in ranked chunks	R@5 = 0.463	Retriever quality
Fallback rate	(ambiguous + incorrect) / 200	17.5 %	How often the KG is used
Cost	wall-clock seconds, prompt + generated tokens	Table 6.6	Efficiency

6.3 Results and Discussion

T1 — Accuracy for no retrieval, standard RAG and corrective RAG

Table 6.2 (T1): Answer quality on 200 CRAG questions, mean ± std over 3 seeds for no retrieval; RAG methods 1 seed (%)

Method	Accuracy %	Missing %	Hallucination %	CRAG score %	EM %	F1 %
No retrieval [BASELINE]	*17.3 ± 1.4*	*17.2 ± 0.3*	*65.5 ± 1.3*	*-48.2 ± 2.8*	*8.8 ± 0.6*	*15.1 ± 0.2*
Standard RAG (top-5)	17.0 (1 seed)	11.5 (1 seed)	71.5 (1 seed)	-54.5 (1 seed)	7.0 (1 seed)	14.8 (1 seed)
Corrective RAG + KG fallback	18.0 (1 seed)	13.5 (1 seed)	68.5 (1 seed)	-50.5 (1 seed)	7.5 (1 seed)	16.5 (1 seed)

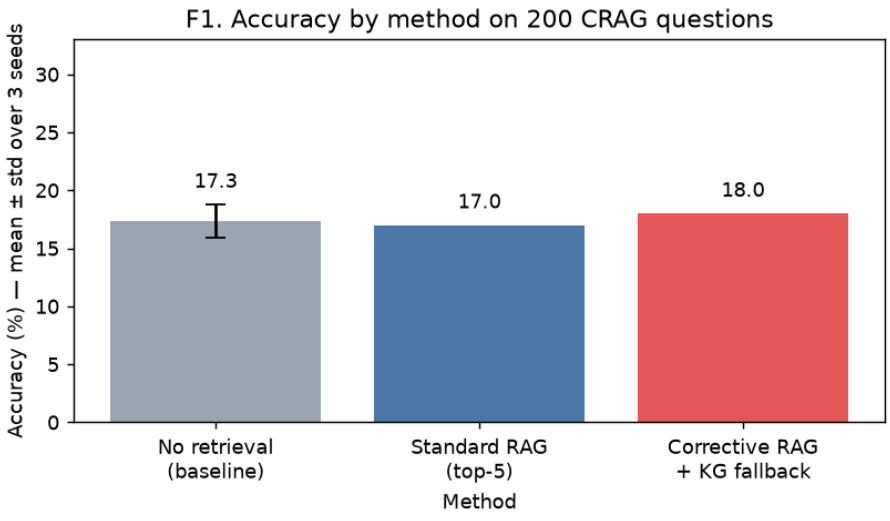


Figure F1: Accuracy by method (error bar = std over 3 seeds, no-retrieval only)

Standard RAG raised accuracy from 17.3 % (no retrieval) to 17.0 % (-0.3 points). Corrective RAG with the KG fallback reached 18.0 % (+1.0 points versus standard RAG). No retrieval was run with 3 seeds (std 1.4 points); because of the GPU time budget the two RAG methods were run with one seed, so their seed variance is not

measured. Using the no-retrieval std as a guide, the difference between the two RAG variants is within sampling noise.

Hallucination rates were 65.5 %, 71.5 % and 68.5 %, and missing ('I don't know') rates 17.2 %, 11.5 % and 13.5 %. Because CRAG's score subtracts hallucinations, a method that abstains more can score better even with similar accuracy. Absolute accuracy is low for every method: a 1.5 B model, strict string-containment scoring and the fact that only 56.5 % of answerable questions have their answer anywhere in the five pages all cap performance.

T2 — Evaluator label counts and fallback rate

Table 6.3 (T2): Evaluator labels and KG fallback rate (thresholds upper 0.60, lower 0.15)

Evaluator label	Count	Share (%)
correct	165	82.5
ambiguous	18	9.0
incorrect	17	8.5
KG fallback (ambiguous+incorrect)	35	17.5
KG entity found (of fallbacks)	29	82.9

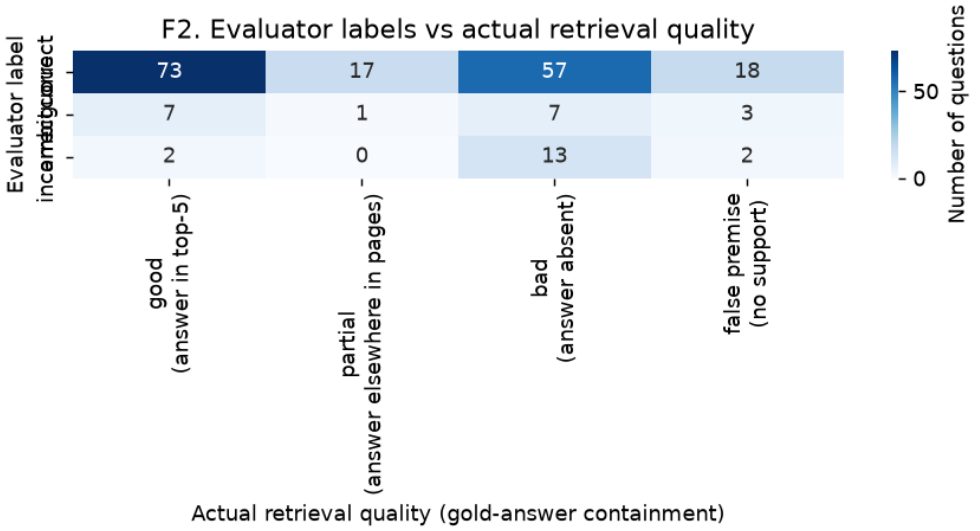


Figure F2: Evaluator labels against actual retrieval quality

The evaluator labelled 165 questions correct, 18 ambiguous and 17 incorrect, so the KG fallback was used for 35 questions (17.5 %). Wikidata returned at least one linked entity for 82.9 % of these.

Figure F2 shows that the evaluator is lenient: when it says 'correct', the top-5 chunks actually contain the gold answer in only 44.2 % of cases, while it labels 89.0 % of truly good retrievals as correct. The cross-encoder scores topical relevance (the page is about the right entity) rather than answer presence, so many questions whose pages are on-topic but lack the specific fact are never routed to the KG. On the questions that were routed (ambiguous + incorrect), accuracy was ambiguous: standard RAG 16.7 % vs corrective 16.7 %; incorrect: standard RAG 5.9 % vs corrective 17.6 %.

Variant analysis — fallback rate and accuracy by question type

Table 6.4: Fallback rate and accuracy (%) by question type (no retrieval: mean ± std of 3 seeds; RAG: 1 seed)

Question type	n	fallback rate %	incorrect %	No retrieval	Standard RAG (top-5)	Corrective RAG + KG fallback
comparison	23	26.1	21.7	29.0 ± 9.1	21.7	26.1

simple	56	23.2	14.3	7.7 ± 2.7	19.6	19.6
false_premise	23	21.7	8.7	66.7 ± 5.0	21.7	26.1
simple_w_condition	29	20.7	0.0	6.9 ± 3.4	17.2	17.2
aggregation	24	12.5	4.2	4.2 ± 0.0	4.2	4.2
multi-hop	18	11.1	5.6	9.3 ± 6.4	5.6	5.6
set	18	0.0	0.0	16.7 ± 0.0	27.8	27.8
post-processing	9	0.0	0.0	7.4 ± 6.4	11.1	11.1

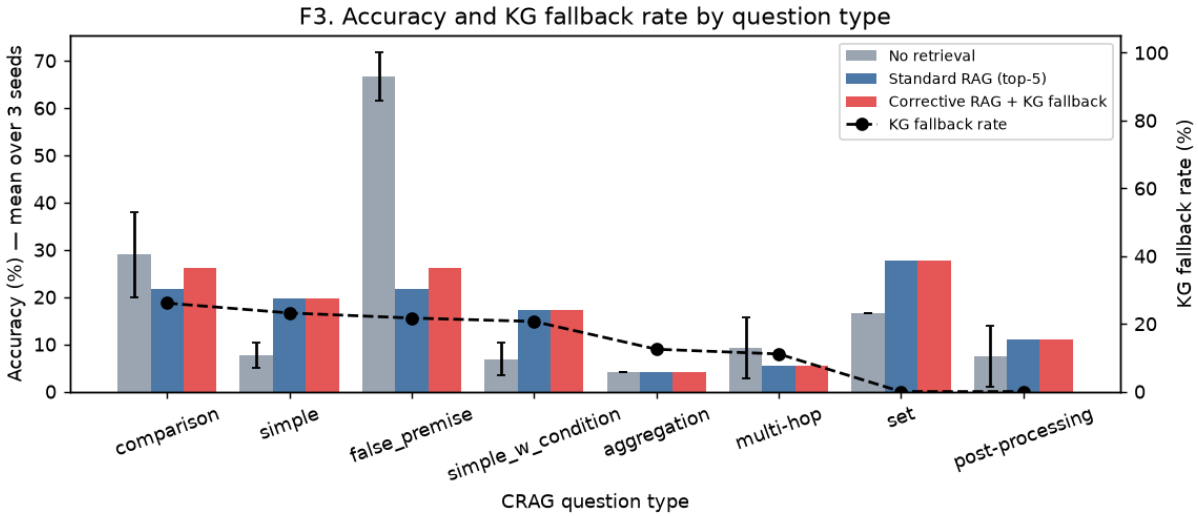


Figure F3: Accuracy (bars) and KG fallback rate (line) by question type

Fallback rate differs strongly by question type: it is highest for comparison (26.1 %) and lowest for set (0.0 %). Relative to standard RAG, corrective RAG helped most on comparison (+4.3 points) and hurt most on aggregation (+0.0 points).

False-premise questions are answered best without retrieval (66.7 %) — retrieved pages about a real entity make the model accept the false premise (standard RAG 21.7 %, corrective 26.1 %). Aggregation, multi-hop and post-processing questions remain hard for every method because a 1.5 B model must combine several facts; a KG triple list rarely contains the pre-computed aggregate. Per-type cells hold only 9–56 questions, so differences under about 5 points should not be over-interpreted.

Evaluator threshold sensitivity

Table 6.5: Label counts and precision of the 'correct' label as thresholds vary (no re-generation)

upper	lower	correct	ambiguous	incorrect	fallback_rate	correct_precision
0.4	0.05	177.0	11.0	12.0	0.115	0.441
0.4	0.15	177.0	6.0	17.0	0.115	0.441
0.4	0.25	177.0	2.0	21.0	0.115	0.441
0.5	0.05	171.0	17.0	12.0	0.145	0.444
0.5	0.15	171.0	12.0	17.0	0.145	0.444
0.5	0.25	171.0	8.0	21.0	0.145	0.444
0.6	0.05	165.0	23.0	12.0	0.175	0.442
0.6	0.15	165.0	18.0	17.0	0.175	0.442
0.6	0.25	165.0	14.0	21.0	0.175	0.442

0.7	0.05	156.0	32.0	12.0	0.22	0.449
0.7	0.15	156.0	27.0	17.0	0.22	0.449
0.7	0.25	156.0	23.0	21.0	0.22	0.449
0.8	0.05	148.0	40.0	12.0	0.26	0.466
0.8	0.15	148.0	35.0	17.0	0.26	0.466
0.8	0.25	148.0	31.0	21.0	0.26	0.466

With the chosen thresholds (0.60 / 0.15) the fallback rate is 17.5 %. Raising the upper threshold to 0.80 increases the fallback rate to 26.0 % and the precision of the 'correct' label changes from 0.442 to 0.466; a stricter evaluator would send more questions to the KG, at the cost of discarding some good web evidence. Thresholds were not tuned on the test questions, to avoid overfitting a 200-question set.

Retrieval quality and cost

Table 6.6a: Retrieval metrics over 177 answerable questions (relevant = chunk contains gold answer)

Retriever	Recall@5	Recall@10	MRR	NDCG@10
BGE-small dense	0.463	0.514	0.408	0.335

Table 6.6: Cost — wall-clock time (s) and tokens per stage

stage	time_s	prompt_tokens	generated_tokens	per_question_tokens
HTML parsing + chunking	62.4	0	0	0.0
BGE-small embedding + retrieval	121.0	0	0	0.0
Evaluator (BGE-reranker-base)	25.9	0	0	0.0
KG lookup (entity extraction + Wikidata)	210.5	2699	209	15.0
Generation: No retrieval	171.6	74862	4881	133.0
Generation: Standard RAG (top-5)	588.9	138362	2202	703.0
Generation: Corrective RAG + KG fallback	579.0	121401	2156	618.0

Total measured time was 29.3 min on an RTX 3050 (4 GB). Generation dominates. Corrective RAG used 121,401 prompt tokens versus 138,362 for standard RAG, because knowledge refinement drops low-score chunks; the KG lookup added 210 s (mostly polite Wikidata rate-limiting) and 2,908 tokens for entity extraction.

Five failure cases

Case 1 (simple). Q: "can you tell me who directed guns n' roses - live in paris?" Gold: "paul becher". Evaluator: ambiguous (max score 0.20); KG entities: Guns N' Roses, Live In Paris, Guns N' Roses; top KG fact: (Guns N' Roses, country of origin, United States). Standard RAG: "I don't know.". Corrective RAG: "Tommy LiPuma". Diagnosis: entity-linking error in the KG.

Case 2 (false_premise). Q: "what is the name of millie bobby brown's upcoming fashion line?" Gold: "invalid question". Evaluator: correct (max score 0.97). Standard RAG: "Florence by Mills". Corrective RAG: "Florence by Mills". Diagnosis: false premise accepted.

Case 3 (multi-hop). Q: "how many grand slams has novak djokovic won on clay courts?" Gold: "3". Evaluator: correct (max score 1.00). Standard RAG: "18". Corrective RAG: "18". Diagnosis: reasoning over evidence.

Case 4 (post-processing). Q: "what was the average amount spent by amazon on their three largest acquisitions to date?" Gold: "~$8.68 billion". Evaluator: correct (max score 0.98). Standard RAG: "To calculate the average amount spent on the three largest acquisitions, we need more specific information about which three acquisitions

we". Corrective RAG: "To calculate the average amount spent on the three largest acquisitions, we need more specific information about which three acquisitions we". Diagnosis: evaluator over-confident.

6.4 Validation / Demonstration Evidence

Demo steps: open 67_P3_Variant_CorrectiveRAG_KG.ipynb in Google Colab (GPU runtime) or locally, run all cells; the notebook downloads the CRAG subsets, uses the cached Wikidata responses, and writes every table and figure to results/ and figures/. The executed notebook (with outputs) and notebook_render.html are included as evidence; screenshots in Section 5.4 and Appendix A are taken from that executed notebook. No credentials or API keys are needed.

7. Sustainability and SDG Alignment

The project's impacts are mostly indirect: it is a research prototype, not a deployed service.

7.1 Sustainability Assessment

Table 7.1: Sustainability assessment

Dimension	Questions to Consider	Project-Specific Assessment / Evidence
Environmental	Energy and compute use; e-waste; resource efficiency	Positive/neutral: all runs on one 60 W laptop GPU; total measured compute ≈ 29 min. Small models (≤1.5 B) and caching avoid repeated API and GPU work.
Social	Accessibility; safety; privacy; fairness	Positive: abstaining ("I don't know") and false-premise detection reduce misinformation. Risk: KG coverage is biased toward well-documented (Western, English) entities.
Economic	Development and operating cost; long-term viability	Positive: ₹0 software/API cost; Wikidata replaces paid web-search APIs used by the original corrective RAG.

7.2 UN Sustainable Development Goals (SDGs)

Table 7.2: SDG alignment

SDG No. and Name	Project Feature / Activity Linked to SDG	Expected or Observed Contribution	Evidence / Indicator
SDG 4: Quality Education	Open, low-cost QA assistant that abstains when unsure	More trustworthy study aid for students without paid tools	Hallucination rate per method (Table 6.2)
SDG 9: Industry, Innovation and Infrastructure	Reproducible open-model RAG on a 4 GB GPU with a public KG	Lowers the barrier to building reliable AI infrastructure	Runs end to end on free compute; cost in Table 6.6

7.3 Sustainability Measures and Trade-offs

Measures: small models in fp16, freeing the reranker before generation, caching every Wikidata call, and stopping early at 48 new tokens. Trade-off: a 1.5 B generator is cheap but weaker at reasoning (aggregation, multi-hop), which limits accuracy.

7.4 Sustainability Indicator / Follow-up

Measured: wall-clock time and tokens per stage (Table 6.6). Proposed future measure: energy per answered question (Wh) using nvidia-smi power logging, and accuracy per 1,000 tokens.

8. Feasibility, Limitations and Risk Considerations

8.1 Feasibility

Technically feasible on free resources: the whole experiment fits a 4 GB GPU or a Colab T4. Operationally, a re-run takes under an hour with cached KG responses. Cost estimate:

Table 8.1: Budget estimate

Item / Resource	Cost (₹)	Basis / Notes
Hardware / hosting / cloud	0	Existing laptop GPU or free Colab
Software / API / dataset	0	Open models, Wikidata API, CRAG (CC BY-NC 4.0)
Other resources	0	—
Total estimated cost	0	No recurring cost; electricity only

8.2 Limitations

- Only 200 questions; per-type cells contain 8–56 questions, so per-type differences of a few points are within noise.
- Automatic string-containment scoring underestimates correct paraphrases and long or numeric answers; no LLM judge or human labels.
- Wikidata is not the CRAG mock KG: finance prices and recent sports results are mostly absent, and facts are as of today, not query_time.
- Entity linking takes only the top Wikidata hit, so ambiguous names link to the wrong entity (see failure cases).
- The 1.5 B generator and 600-character document cap limit reasoning over long tables; thresholds were not tuned on a dev split.
- Because of GPU time (a thermally throttled 4 GB laptop GPU), only the no-retrieval baseline was run with 3 seeds; the RAG methods used seed 0, so their variance is not measured. Retrieval, evaluator and KG lookup are deterministic.

8.3 Risks, Ethics, Privacy and Security

No personal data is processed. The dataset licence (CC BY-NC 4.0) restricts use to non-commercial research. Wikidata usage follows the Wikimedia API etiquette (descriptive User-Agent, throttling, back-off). Risks: confident wrong answers from a small model, KG bias toward well-documented entities, and over-trust in the evaluator. Safeguards: abstention instruction, reporting hallucination separately, and publishing the evaluator scores and KG facts used for every question (evaluator_and_kg_trace.json).

9. Conclusion and Future Scope

9.1 Summary

A training-free corrective-RAG pipeline with a Wikidata KG-lookup fallback was built and evaluated on 200 stratified CRAG questions with open models on a 4 GB GPU. Accuracy was 17.3 % without retrieval, 17.0 % with standard RAG and 18.0 % with corrective RAG (baseline: mean of 3 seeds; RAG methods: seed 0). The evaluator routed 17.5 % of questions to the KG, with the highest fallback rate on comparison questions.

The main finding is that the bottleneck is the evaluator, not the fallback: a relevance cross-encoder judges topical match and therefore rarely detects that on-topic pages lack the answer; and when the KG is used, top-1 entity linking and Wikidata's limited coverage of finance and recent sports facts limit gains. All objectives were met,

including negative results, which are reported as measured. The approach costs nothing beyond a laptop GPU, supporting SDG 4 and SDG 9.

9.2 Future Scope

- Replace top-1 entity linking with question-aware linking (type constraints, multiple candidates) and SPARQL property paths.
- Use the CRAG mock-KG API for finance/sports, and filter KG facts by query_time.
- Tune evaluator thresholds on a held-out split and compare with the Primary's thresholds; try an LLM-based evaluator.
- Add an LLM judge / human labels (as in project P9) to reduce scoring bias; scale to the full 2.7 k public questions.
- Measure energy per answer and add decompose-then-recompose refinement from the original corrective RAG.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. NeurIPS, 2020. arXiv:2005.11401.

[2] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective Retrieval Augmented Generation," arXiv:2401.15884, 2024.

[3] X. Yang et al., "CRAG – Comprehensive RAG Benchmark," in Proc. NeurIPS Datasets and Benchmarks Track, 2024. arXiv:2406.04744.

[4] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in Proc. ICLR, 2024.

[5] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packed Resources for General Chinese Embeddings," in Proc. SIGIR, 2024 (BGE models).

[6] D. Vrandečić and M. Krötzsch, "Wikidata: A Free Collaborative Knowledgebase," Communications of the ACM, vol. 57, no. 10, pp. 78–85, 2014.

[7] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying Large Language Models and Knowledge Graphs: A Roadmap," IEEE TKDE, vol. 36, no. 7, 2024.

[8] D. Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," arXiv:2404.16130, 2024.

[9] Qwen Team, "Qwen2.5 Technical Report," arXiv:2412.15115, 2024.

[10] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in Proc. EACL (Demos), 2024.

[11] Quivr, "CRAG (stratified subsets of Meta CRAG)," Hugging Face Datasets, 2024. https://huggingface.co/datasets/Quivr/CRAG (accessed 02 Nov 2026).

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP, 2019.

[13] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of IR Models," in Proc. NeurIPS Datasets and Benchmarks, 2021.

[14] K. Järvelin and J. Kekäläinen, "Cumulated Gain-Based Evaluation of IR Techniques," ACM TOIS, vol. 20, no. 4, pp. 422–446, 2002.

[15] Wikimedia Foundation, "Wikidata Action API and API etiquette," https://www.wikidata.org/w/api.php (accessed 02 Nov 2026).

Appendices

Appendix A: Additional screenshots / outputs

In [3]:
```
ex = sample[0]
print({k: v for k, v in ex.items() if k != "search_results"})
print("pages:", [p["page_name"][:60] for p in ex["search_results"]])
```

```
{'interaction_id': '8d194198-f9a6-4988-a509-d3c138af56a6', 'query_time': '03/19/2024, 23:47:33 PT', 'domain': 'movie', 'question_type': 'simple', 'stat
ic_or_dynamic': 'static', 'query': 'who won the oscar best actor in 2019?', 'answer': 'rami malek', 'split': 1}
pages: ["LIST: Every actor that has won 'Best Actor' at the Academy A", 'Oscar Best Actor Gallery: Every Winner in Academy Award Hist', 'Academy Award
for Best Actor - Wikipedia', "LIST: Every actor that has won 'Best Actor' at the Academy A", 'Oscar Best Actor Gallery: Every Winner in Academy Award H
ist']
```

Figure A1: Example CRAG record and its five search-result pages

In [4]:
```
CHUNK, OVERLAP, MAX_WORDS_PAGE = 200, 40, 6000

def html_text(html):
    soup = BeautifulSoup(html, "lxml")
    for t in soup(["script", "style", "noscript", "header", "footer", "nav"]): t.decompose()
    return re.sub(r"\s+", " ", soup.get_text(" ")).strip()

def chunk_words(text):
    w = text.split()[:MAX_WORDS_PAGE]
    return [" ".join(w[i:i + CHUNK]) for i in range(0, max(len(w) - OVERLAP, 1), CHUNK - OVERLAP)]

t0 = time.time()
for r in sample:
    r["chunks"] = []
    for p in r["search_results"]:
        body = html_text(p["page_result"]) if p["page_result"] else ""
        r["chunks"] += [f"{p['page_name']}: {c}" for c in chunk_words(p["page_snippet"] + " " + body) if c]
    del r["search_results"]                                  # free ~400 MB of raw HTML
parse_time = time.time() - t0
n_chunks = [len(r["chunks"]) for r in sample]
print(f"parsed in {parse_time:.1f}s | chunks total={sum(n_chunks)} mean/q={np.mean(n_chunks):.1f}")
```

```
C:\Users\Manish Raje\AppData\Local\Temp\ipykernel_5292\3434445447.py:4: XMLParsedAsHTMLWarning: It looks like you're using an HTML parser to parse an X
ML document.

Assuming this really is an XML document, what you're doing might work, but you should know that using an XML parser will be more reliable. To parse thi
s document as XML, make sure you have the Python package 'lxml' installed, and pass the keyword argument `features="xml"` into the BeautifulSoup constr
uctor.

If you want or need to use an HTML parser on this document, you can make this warning go away by filtering it. To do that, run this code before calling
the BeautifulSoup constructor:

    from bs4 import XMLParsedAsHTMLWarning
    import warnings

    warnings.filterwarnings("ignore", category=XMLParsedAsHTMLWarning)

  soup = BeautifulSoup(html, "lxml")
parsed in 62.4s | chunks total=13036 mean/q=65.2
```

Figure A2: HTML parsing and chunking statistics

In [5]:
```
from sentence_transformers import SentenceTransformer, CrossEncoder
emb = SentenceTransformer(EMBEDDER, device=DEVICE)
QPFX = "Represent this sentence for searching relevant passages: "
K = 10
t0 = time.time()
for r in sample:
    cv = emb.encode(r["chunks"], batch_size=64, normalize_embeddings=True)
    qv = emb.encode(QPFX + r["query"], normalize_embeddings=True)
    order = np.argsort(-(cv @ qv))[:K]
    r["ret"] = [r["chunks"][i] for i in order]
embed_time = time.time() - t0
print(f"embedding + retrieval: {embed_time:.1f}s")
```

```
D:\KGIRs\67_P3_Variant\.venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipy
widgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
embedding + retrieval: 121.0s
```

Figure A3: BGE-small embedding and retrieval timing

In [8]:
```
from transformers import AutoTokenizer, AutoModelForCausalLM
tok = AutoTokenizer.from_pretrained(LLM, padding_side="left")
llm = AutoModelForCausalLM.from_pretrained(LLM, dtype=torch.float16 if DEVICE == "cuda" else torch.float32).to(DEVICE).eval()
TOKENS = collections.Counter()

def chat(prompts, max_new=48, seed=None, stage="gen", bs=4):
    outs = []
    for i in range(0, len(prompts), bs):
        msgs = [tok.apply_chat_template([{"role": "user", "content": p}], tokenize=False, add_generation_prompt=True) for p in prompts[i:i + bs]]
        enc = tok(msgs, return_tensors="pt", padding=True, truncation=True, max_length=1100).to(DEVICE)
        kw = dict(do_sample=True, temperature=0.7, top_p=0.9) if seed is not None else dict(do_sample=False)
        if seed is not None: torch.manual_seed(seed * 1000 + i)
        with torch.no_grad():
            g = llm.generate(**enc, max_new_tokens=max_new, pad_token_id=tok.eos_token_id, **kw)
        new = g[:, enc["input_ids"].shape[1]:]
        TOKENS[stage + "_in"] += int(enc["attention_mask"].sum()); TOKENS[stage + "_out"] += int((new != tok.eos_token_id).sum())
        outs += [o.strip() for o in tok.batch_decode(new, skip_special_tokens=True)]
    return outs

print(chat(["What is the capital of France? Answer in a few words."]))
```

```
The following generation flags are not valid and may be ignored: ['temperature', 'top_p', 'top_k']. Set `TRANSFORMERS_VERBOSITY=info` for more details.
['Paris']
```

Figure A4: Generator loaded and sanity-checked

In [10]:
```
SYS = ("You are a careful question-answering assistant. The question was asked on {qt}. Answer in as few words as possible. "
       "If the question contains a false premise, answer exactly 'invalid question'. If you are not sure, answer exactly 'I don't know'.\n")

def p_none(r): return SYS.format(qt=r["query_time"]) + f"Question: {r['query']}\nAnswer:"
def p_rag(r, docs, facts=()):
    ctx = "".join(f"[Doc {i+1}] {d[:400]}\n" for i, d in enumerate(docs))
    kg = "".join(f"- {f}\n" for f in facts)
    return (SYS.format(qt=r["query_time"]) + (f"Web documents:\n{ctx}" if docs else "") +
            (f"Knowledge-graph facts (Wikidata):\n{kg}" if kg else "") + f"Question: {r['query']}\nAnswer:")

def p_crag(r):
    if r["label"] == "correct":   return p_rag(r, r["refined"])
    if r["label"] == "incorrect": return p_rag(r, [], r["kg_facts"]) if r["kg_facts"] else p_none(r)
    return p_rag(r, r["refined"], r["kg_facts"])

METHODS = {"No retrieval": p_none, "Standard RAG (top-5)": lambda r: p_rag(r, r["ret"][:5]), "Corrective RAG + KG fallback": p_crag}
preds, gen_time = {}, {}
for m, fn in METHODS.items():
    prompts = [fn(r) for r in sample]
    for s in (GEN_SEEDS if m == "No retrieval" else RAG_SEEDS):
        t0 = time.time(); preds[(m, s)] = chat(prompts, seed=s, stage=m); gen_time[(m, s)] = time.time() - t0
        print(f"{m:30s} seed {s}: {gen_time[(m, s)]:.1f}s")
```

```
No retrieval                   seed 0: 55.7s
No retrieval                   seed 1: 52.6s
No retrieval                   seed 2: 63.3s
Standard RAG (top-5)           seed 0: 588.9s
Corrective RAG + KG fallback   seed 0: 579.0s
```

Figure A5: Answer generation for the three methods and three seeds (timings)

In [12]:
```
lab_counts = pd.DataFrame({"count": lab, "share %": (100 * lab / len(sample)).round(1)})
lab_counts.loc["KG fallback (ambiguous+incorrect)"] = [lab["ambiguous"] + lab["incorrect"], round(100 * fallback_rate, 1)]
lab_counts.loc["KG entity found (of fallbacks)"] = [sum(bool(r["kg_found"]) for r in fb), round(100 * np.mean([bool(r["kg_found"]) for r in fb]), 1)]
print(lab_counts)
```

```
                                  count  share %
correct                             165     82.5
ambiguous                            18      9.0
incorrect                            17      8.5
KG fallback (ambiguous+incorrect)    35     17.5
KG entity found (of fallbacks)       29     82.9
```

Figure A6: T2 evaluator label counts and fallback rate output

In [13]:
```
qt = res.groupby(["qtype", "method", "seed"])["acc"].mean().groupby(["qtype", "method"]).agg(["mean", "std"]).unstack("method")
fb_by = pd.DataFrame([dict(qtype=r["question_type"], fallback=r["label"] != "correct", incorrect=r["label"] == "incorrect") for r in sample]).groupby(
T3 = pd.DataFrame({"n": fb_by.size(), "fallback rate %": (100 * fb_by["fallback"].mean()).round(1), "incorrect %": (100 * fb_by["incorrect"].mean()).r
for m in METHODS:
    T3[m + " acc %"] = [f"{100*a:.1f} ± {100*b:.1f}" for a, b in zip(qt[("mean", m)], qt[("std", m)])]
T3 = T3.sort_values("fallback rate %", ascending=False)
print(T3.to_string())

# accuracy of corrective RAG split by evaluator branch
br = res[res.method.isin(["Standard RAG (top-5)", "Corrective RAG + KG fallback"])].groupby(["label", "method", "seed"])["acc"].mean().groupby(["label
print((100 * br).round(1))
```

```
                     n  fallback rate %  incorrect % No retrieval acc % Standard RAG (top-5) acc % Corrective RAG + KG fallback acc %
qtype
comparison          23             26.1         21.7        29.0 ± 9.1               21.7 ± nan                       26.1 ± nan
simple              56             23.2         14.3         7.7 ± 2.7               19.6 ± nan                       19.6 ± nan
false_premise       23             21.7          8.7        66.7 ± 5.0               21.7 ± nan                       26.1 ± nan
simple_w_condition  29             20.7          0.0         6.9 ± 3.4               17.2 ± nan                       17.2 ± nan
aggregation         24             12.5          4.2         4.2 ± 0.0                4.2 ± nan                        4.2 ± nan
multi-hop           18             11.1          5.6         9.3 ± 6.4                5.6 ± nan                        5.6 ± nan
set                 18              0.0          0.0        16.7 ± 0.0               27.8 ± nan                       27.8 ± nan
post-processing      9              0.0          0.0         7.4 ± 6.4               11.1 ± nan                       11.1 ± nan
method     Corrective RAG + KG fallback  Standard RAG (top-5)
label
ambiguous                          16.7                  16.7
correct                            18.2                  18.2
incorrect                          17.6                   5.9
```

Figure A7: Fallback rate and accuracy by question type (Variant analysis)

In [17]:
```
mx = np.array([max(r["ev_scores"]) for r in sample])
sens = []
for up in (0.4, 0.5, 0.6, 0.7, 0.8):
    for lo in (0.05, 0.15, 0.25):
        sens.append(dict(upper=up, lower=lo, correct=int((mx >= up).sum()), ambiguous=int(((mx < up) & (mx >= lo)).sum()), incorrect=int((mx < lo).sum
                         fallback_rate=round(float((mx < up).mean()), 3), correct_precision=round(float(good[mx >= up].mean()) if (mx >= up).any() els
T4 = pd.DataFrame(sens); print(T4.to_string(index=False))
```

```
upper  lower  correct  ambiguous  incorrect  fallback_rate  correct_precision
  0.4   0.05      177         11         12          0.115              0.441
  0.4   0.15      177          6         17          0.115              0.441
  0.4   0.25      177          2         21          0.115              0.441
  0.5   0.05      171         17         12          0.145              0.444
  0.5   0.15      171         12         17          0.145              0.444
  0.5   0.25      171          8         21          0.145              0.444
  0.6   0.05      165         23         12          0.175              0.442
  0.6   0.15      165         18         17          0.175              0.442
  0.6   0.25      165         14         21          0.175              0.442
  0.7   0.05      156         32         12          0.220              0.449
  0.7   0.15      156         27         17          0.220              0.449
  0.7   0.25      156         23         21          0.220              0.449
  0.8   0.05      148         40         12          0.260              0.466
  0.8   0.15      148         35         17          0.260              0.466
  0.8   0.25      148         31         21          0.260              0.466
```

Figure A8: Threshold sensitivity table

In [18]:
```
cost = pd.DataFrame([
    ("HTML parsing + chunking", parse_time, 0, 0),
    ("BGE-small embedding + retrieval", embed_time, 0, 0),
    ("Evaluator (BGE-reranker-base)", eval_time, 0, 0),
    ("KG lookup (entity extraction + Wikidata)", kg_time, TOKENS["kg_entity_in"], TOKENS["kg_entity_out"]),
] + [(f"Generation: {m}", sum(v for (mm, s), v in gen_time.items() if mm == m), TOKENS[m + "_in"], TOKENS[m + "_out"]) for m in METHODS],
    columns=["stage", "time_s", "prompt_tokens", "generated_tokens"])
cost["time_s"] = cost["time_s"].round(1)
cost["per_question_tokens"] = ((cost.prompt_tokens + cost.generated_tokens) / (len(sample) * np.where(cost.stage == "Generation: No retrieval", 3, 1))
print(cost.to_string(index=False))
```

```
                                stage  time_s  prompt_tokens  generated_tokens  per_question_tokens
                HTML parsing + chunking    62.4              0                 0                  0.0
        BGE-small embedding + retrieval   121.0              0                 0                  0.0
          Evaluator (BGE-reranker-base)    25.9              0                 0                  0.0
KG lookup (entity extraction + Wikidata)   210.5           2699               209                 15.0
                  Generation: No retrieval   171.6          74862              4881                133.0
       Generation: Standard RAG (top-5)   588.9         138362              2202                703.0
Generation: Corrective RAG + KG fallback   579.0         121401              2156                618.0
```

Figure A9: Cost table: time and tokens per stage

In [19]:
```
T1_show.to_csv(os.path.join(OUT, "T1_accuracy_by_method.csv")); per_seed.to_csv(os.path.join(OUT, "metrics_per_seed.csv"))
lab_counts.to_csv(os.path.join(OUT, "T2_evaluator_labels.csv")); T3.to_csv(os.path.join(OUT, "T3_by_question_type.csv"))
T4.to_csv(os.path.join(OUT, "T4_threshold_sensitivity.csv"), index=False); cost.to_csv(os.path.join(OUT, "T5_cost.csv"), index=False)
ret_table.to_csv(os.path.join(OUT, "retrieval_metrics.csv")); cm.to_csv(os.path.join(OUT, "F2_confusion_counts.csv"))
long = per_seed.reset_index().melt(id_vars=["method", "seed"], var_name="metric")
long = pd.concat([long, ret_table.reset_index().melt(id_vars="index", var_name="metric").rename(columns={"index": "method"}).assign(seed="n/a (determi
long.to_csv(os.path.join(OUT, "all_metrics.csv"), index=False)
json.dump([{k: r[k] for k in ("interaction_id", "query", "answer", "question_type", "label", "ev_scores", "kg_entities", "kg_facts") if k in r} for r
          open(os.path.join(OUT, "evaluator_and_kg_trace.json"), "w"), indent=1)

s0 = res[res.seed == 0].pivot(index="id", columns="method", values=["acc", "pred"])
meta = {r["interaction_id"]: r for r in sample}
fails = [i for i in s0.index if s0.loc[i, ("acc", "Standard RAG (top-5)")] and not s0.loc[i, ("acc", "Corrective RAG + KG fallback")]][:3]
fails += [i for i in s0.index if not s0.loc[i, ("acc", "Corrective RAG + KG fallback")] and meta[i]["label"] == "correct"][:2]
for i in fails:
    r = meta[i]
    print(f"\n[{r['question_type']}] {r['query']}\n gold: {r['answer']} | evaluator: {r['label']} (max={max(r['ev_scores']):.2f}) | KG entities: {r.ge
    print(" std RAG :", s0.loc[i, ("pred", "Standard RAG (top-5)")]); print(" CRAG+KG :", s0.loc[i, ("pred", "Corrective RAG + KG fallback")])
    if r["kg_facts"]: print(" top KG fact:", r["kg_facts"][0])
print("\nDone.")
```

```
[false_premise] what is the name of millie bobby brown's upcoming fashion line?
 gold: invalid question | evaluator: correct (max=0.97) | KG entities: None
 std RAG : Florence by Mills
 CRAG+KG : Florence by Mills

[multi-hop] how many grand slams has novak djokovic won on clay courts?
 gold: 3 | evaluator: correct (max=1.00) | KG entities: None
 std RAG : 18
 CRAG+KG : 18

Done.
```

Figure A10: Results saved and failure cases printed